

Algoritmos y Estructuras de Datos

Estructuras dinámicas en memoria

Estructuras dinámicas en memoria

Una estructura de datos es un conjunto de datos almacenados en algún lado. Un archivo de datos es una estructura de datos por ejemplo. Mismo los arreglos, vectores y matrices.

Las estructuras dinámicas son estructuras que pueden crecer y decrecer durante la ejecución del programa. Las estructuras de datos estáticas siempre van a tener un tamaño fijo que se define al principio, y que no cambia durante la ejecución del programa.

Para poder trabajar con las estructuras dinámicas, debemos utilizar la memoria dinámicamente. Cuando yo declaro una variable cualquiera en una estructura estática, reservo un espacio en memoria con el nombre de la variable.

Manejo dinámico de la memoria

Existe un tipo de variable denominada puntero. El contenido que puede tener esa variable es una dirección de memoria de donde se puede almacenar un entero. O sea, no va a tener un entero, sino una dirección de memoria que puede almacenar ese tipo de dato. Por ejemplo, en este caso podemos almacenar en la variable p la dirección de memoria a definida en el programa colocando el &.

```
#include <iostream>

using namespace std;

int main()
{
    int a;
    int* p;
    a=5;
    p=&a;
    return 0;
}
```

Entonces ahora la variable p contiene ahora la dirección de memoria de a. Es decir, la variable p apunta a la variable a. Entonces, en un programa se puede acceder al espacio de memoria a a través de la misma variable o a través de su puntero.

```
cout<<a<<endl;|
cout<<*p<<endl;

return 0;
```

Los espacios de memoria a los que apunta p pueden ser modificados, pues p seguirá apuntando a dicho espacio y mostrará la modificación siempre que se efectúe.

```
a=8;
cout<<*p<<endl;

return 0;
```

Ahora lo que vamos a hacer es trabajar solo con punteros. Definimos el puntero p y pedimos un espacio de memoria para que p pueda apuntar a ese espacio. Para ello tenemos dos funciones:

En C tenemos el malloc. Le tengo que indicar cuanta memoria quiero en bytes y me devuelve una dirección de memoria en un puntero genérico.

En C++ tenemos el new. Le tengo que indicar el tipo de datos para el que yo quiero guardar datos, y me devuelve un puntero específico para ese tipo de datos. Vamos a trabajar con esta última función.

Al puntero p definido previamente le asignamos new int, que lo que hace es buscarme un espacio de memoria disponible para guardarme un entero. El new me va a devolver esa dirección de memoria, por lo que voy a hacer que p apunte a ese espacio de memoria. En ese espacio de memoria puedo guardar un entero. Ahora así puedo guardar a lo apuntado por p un valor, en este caso, 5.

```
int *p, *q;
p=new int;
*p=5;
cout<<*p<<endl;
cout<<p<<endl;

return 0;
```

Vamos a diferenciar p de *p. p es un puntero, o sea, el contenido de la variable p es la dirección de memoria de donde se puede almacenar un entero. *p es un entero, es decir, con esta variable puedo hacer cualquier operación que involucre a un entero.

A la variable p le puedo asignar al valor NULL, que es el valor neutro de los punteros. Es decir, con esto estoy diciendo que p apunta a nada. Así, p deja de apuntar al espacio de memoria que le había asignado. Ahora, lo que estaba puesto en ese espacio sigue estando ahí, pero durante la ejecución ocupa un espacio innecesario, porque nadie apunta ahí. Para liberar ese espacio de memoria debemos usar la función free (en C) o delete (en C++). Esto debo hacerlo antes de asignar NULL a p. Antes de modificarle el puntero a p debo ponerle el NULL si o sí, o en todo caso lo que puedo hacer es volver a pedir memoria.

```
delete p;
p=NULL;

return 0;
```

Puedo asignarle a un puntero otro más poniendo por ejemplo que q=p. De esta manera, q va a guardar el mismo espacio de memoria que p y por ende, va a apuntar a dicho espacio.

```

q=p;
cout<<*q<<endl;
cout<<q<<endl;

return 0;

```

Asimismo, puedo comparar punteros por igual o por distinto, para saber si están apuntando al mismo espacio de memoria o a otro. Con mayor o menor no se puede porque recordemos que los punteros guardan espacios de memoria.

Otra cosa que se puede hacer, por ejemplo, es pedir memoria para q y asignarle a lo que apunta q, el valor que apunta p. De esta manera, creamos un nuevo espacio de memoria y ponemos el mismo valor que esta en p en el espacio que apunta q.

```

q=new int;
*q=*p;
cout<<*q<<endl;
cout<<q<<endl;

return 0;

```

Manejo dinámico de memoria con structs

Ahora vamos a trabajar con structs. La mecánica de los structs sigue siendo la misma, salvo que ahora cuando trabajamos con punteros, debemos pedir memoria con el tipo de variable del struct. Lo que va a cambiar es la forma de acceder a escribir cada miembro del struct:

```

int main()
{
    Alumno*p;
    /*
    Alumno alu;
    p=&alu;
    cin>>alu.legajo;
    cin>>alu.nombre;
    cin>>alu.nota;
    */
    //Alumno alu;

    p=new Alumno;
    //alu=*p;
    |
    cin>>(*p).legajo;
    cin>>(*p).nombre;
    cin>>(*p).nota;

    return 0;
}

```

Existe, además, otra sintaxis para hacer esto mismo:

```

cin>>p->legajo;
cin>>p->nombre;
cin>>p->nota;

return 0;

```

Sin embargo, esto quizás no sea muy eficiente cuando se trabaja con grandes cantidades de variables, ya que se requiere utilizar un puntero por cada uno. Entonces vamos a usar vectores para guardar estas variables.

Uso de vectores, punteros y structs en estructuras dinámicas

Vamos a definir un vector de punteros a alumnos. Cargamos el mismo con la función cargarAlumnos y luego mostramos el promedio de notas con la función prom. Entonces el main queda tal que así:

```

int main()
{
    Alumno*vecAlumnos[3];
    cargarAlumnos(vecAlumnos, 3);
    cout<<prom(vecAlumnos, 3);
    return 0;
}

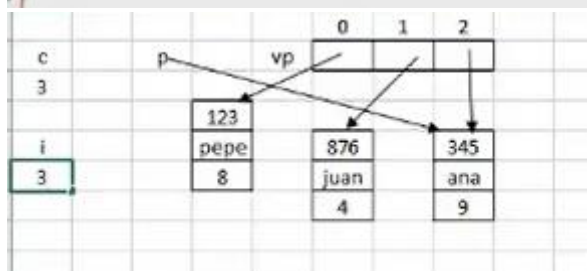
```

En la función cargarAlumnos hacemos un for que vaya de i=0 hasta c, donde definimos un puntero p de tipo alumnos y cargamos el struct con la misma mecánica que se describió anteriormente. Luego, al vector le asignamos el puntero para que éste apunte al espacio de memoria que se llenó con el puntero de p. De esta manera cada posición del vector va a apuntar a un espacio de memoria distinto que se va a ir cargando con p.

```

void cargarAlumnos(Alumno*vp[],int c)
{
    Alumno*p;
    for(int i=0;i<c;i++)
    {
        p=new Alumno;
        cin>>p->legajo;
        cin>>p->nombre;
        cin>>p->nota;
        vp[i]=p;
    }
}

```



Otra forma de hacerlo es sin usar la variable p, directamente usando el vector:

```
for(int i=0;i<c;i++)
{
    vp[i]=new Alumno;
    cin>>vp[i]->legajo;
    cin>>vp[i]->nombre;
    cin>>vp[i]->nota;
}
```

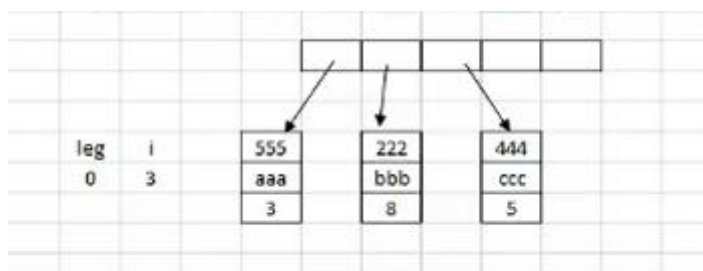
La función promedio va a quedar tal que así:

```
float prom(Alumno*v[],int c)
{
    int s=0;
    for(int i=0;i<c;i++)
        s+=v[i]->nota;
    return s/c;
}
```

Hacer todo esto de la forma que se hizo, **no tiene sentido**, porque es lo mismo que hacerlo con structs solos.

Si yo, por ejemplo, cargo un vector normalmente y le asigno un espacio de 5, y solo ocupo 3, estoy desperdiciando memoria. Además, si en vez de 5 al final quiero ocupar 8, no voy a poder tampoco porque solo me permite 5. Entonces vamos a usar un vector de punteros:

```
int cargarAlumnos(Alumno *vp[],int c)
{
    int leg,i=0;
    cin>>leg;
    while(leg!=0 && i<c)
    {
        vp[i]=new Alumno;
        vp[i]->legajo=leg;
        cin>>vp[i]->nombre;
        cin>>vp[i]->nota;
        i++;
        cin>>leg;
    }
    return i;
}
```

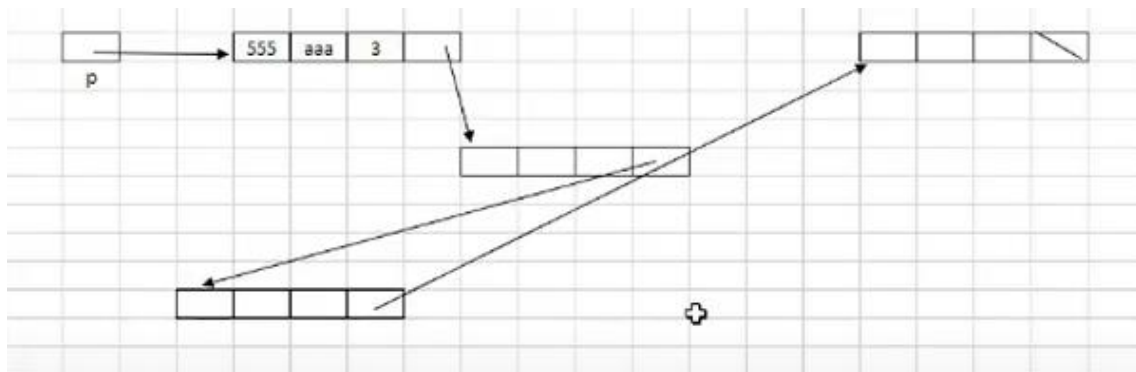


En este caso gano memoria porque tengo tantos alumnos como new use, es decir, 3. Pero nuevamente, si quiero cargar mas de 5 no voy a poder. Esto sigue siendo estático, por más que utilice la memoria de forma dinámica. Entonces, esto no me sirve.

Estructuras dinámicas

En una estructura dinámica necesito tener el puntero a cada uno de los alumnos, es decir, tengo que hacer un new por cada alumno, y además, por cada uno de esos new tengo que guardar esa dirección de memoria en algún lado, porque sino no puedo acceder.

La idea de una estructura dinámica es agregarle al dato un miembro más, donde ese miembro va a guardar la dirección de memoria de otro elemento, es decir, de otro alumno. Entonces cuando creo el segundo pongo la dirección de memoria en el primero. Así, cada elemento tiene la dirección de memoria de otro, salvo el primero, que está en una variable externa p. En el último elemento de todos le ponemos un NULL para que no le quede basura.



Una de las ventajas que tiene respecto a un vector de struct es que no tengo que prever cuanto espacio voy a usar. La desventaja es que ya no puedo hacer acceso directo, es decir, para ir al tercer elemento tengo que arrancar del primero.

En principio vamos a estudiar tres estructuras dinámicas: pilas, colas. Estas dos son estructuras muy básicas donde muchas cosas no se pueden hacer, tienen restricciones. Después vamos a ver las listas, que no tienen limitaciones. Dentro de las listas están las simplemente enlazadas y las listas doblemente enlazadas y las listas circulares. Después vamos a ver la combinación de listas y con vectores. Finalmente, vamos a ver árboles binarios.